Decision Trees for Classification: A Beginner's Guide using the Titanic Dataset

Student Name: Boomiga Mathanavel / Student Id: 24136630

Introduction

When we make decisions, we usually ask ourselves a few questions first. For example, "Is it raining? Should I take an umbrella?" This is basically how a Decision Tree works — it asks questions about the data and follows the answers to make a prediction.

In this tutorial, I will show you how to build a Decision Tree classifier using Python. I am using the Titanic dataset, which contains real information about passengers who were on the Titanic in 1912. The goal is to predict whether a passenger survived or not based on features like their age, gender, and ticket class.

I chose this topic because Decision Trees are one of the easiest machine learning algorithms to understand, and the Titanic dataset is a great way to see how it works in real life. Even if you are new to machine learning, I hope this tutorial helps you get started.

What is a Decision Tree?

A Decision Tree is a supervised machine learning algorithm. It learns from labelled data and makes predictions by asking a series of questions about the features.

The tree has three types of nodes:

- Root node — the first and most important question, at the very top of the tree
- Internal nodes — follow-up questions that keep splitting the data
- Leaf nodes — the final answer, for example "Survived" or "Not Survived"

At each node, the algorithm chooses the feature that best separates the data into groups. It measures this using something called Information Gain, which is based on Entropy — a way of measuring how mixed or uncertain a group of data is. The higher the Information Gain, the better the split.

What I personally like about Decision Trees is that you can actually look at the tree and understand every decision it made. Unlike some other algorithms, it is very easy to explain.

The Dataset

The Titanic dataset has information about 891 passengers. Each row is one passenger, and the column we are trying to predict is `survived` — where 1 means they survived and 0 means they did not.

The features I used are:

- `pclass` — the ticket class (1st, 2nd, or 3rd)
- `sex` — gender, converted to numbers (0 = male, 1 = female)
- `age` — the passenger's age
- `fare` — how much they paid for their ticket

The good thing about this dataset is that it is already built into the seaborn library, so there is no need to download any file.

Step 1 — Load and clean the data

python
```python
import seaborn as sns
df = sns.load_dataset('titanic')
df = df[['survived', 'pclass', 'sex', 'age', 'fare']].dropna()
df['sex'] = df['sex'].map({'male': 0, 'female': 1})
df.head()
```

Here I loaded the dataset and kept only the columns I needed. I also removed rows with missing values using `dropna()`, because the model cannot handle empty cells. The `sex` column was converted from words to numbers since machine learning models only work with numerical data. After cleaning, 714 rows remained.

Step 2 — Split into training and test sets

python
```python
from sklearn.model_selection import train_test_split
X = df[['pclass', 'sex', 'age', 'fare']]
y = df['survived']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

I split the data so that 80% is used for training and 20% for testing. This is important because we want to test the model on data it has never seen before. If we tested it on the same data it trained on, the results would not be reliable — the model would just be memorising the answers.

Step 3 — Train the Decision Tree

python
```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)
```

This is where we actually train the model. I set `max_depth=3`, which means the tree can only ask up to 3 levels of questions. Without this limit, the tree would grow too deep and start memorising the training data instead of learning general patterns. This problem is called overfitting, and limiting the depth is one simple way to avoid it.

Step 4 — Evaluate the model

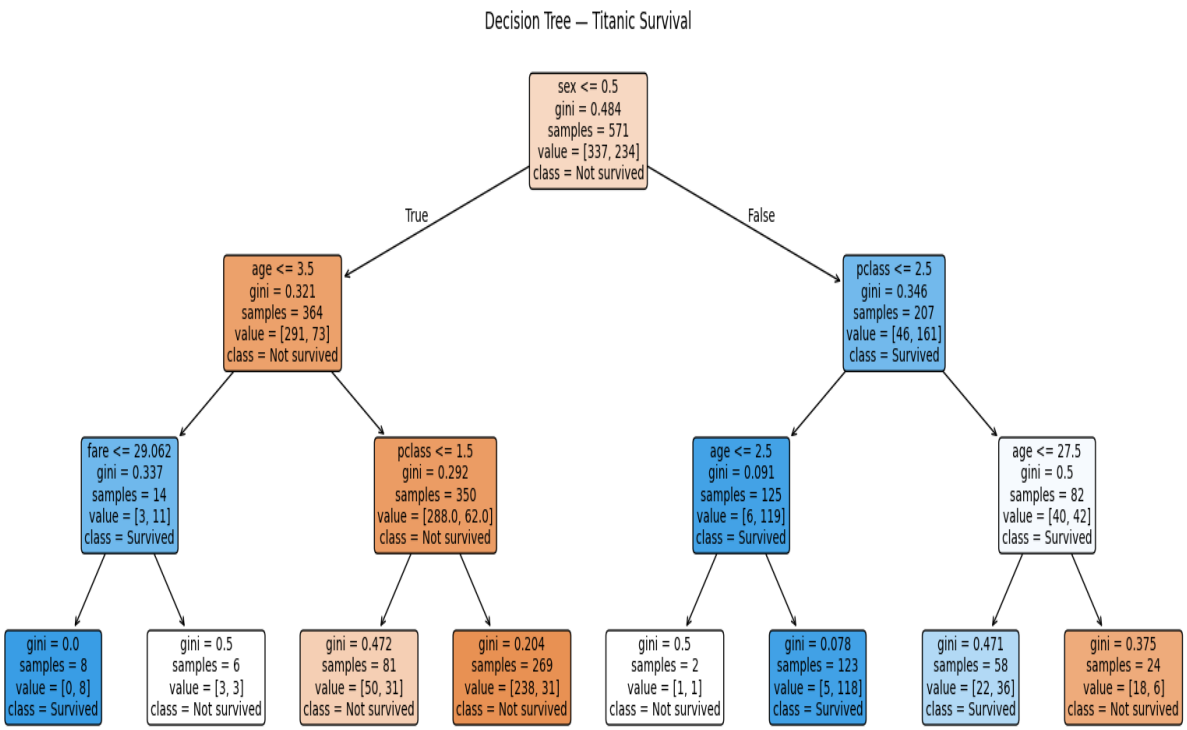
python
```
from sklearn.metrics import accuracy_score, classification_report
y_pred = model.predict(X_test)
print(accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

The model got around 80% accuracy on the test set, which means it correctly predicted the survival outcome for about 8 out of every 10 passengers. I also printed the classification report, which shows more detailed results like precision and recall for each class. This is useful because accuracy alone does not always tell the full story.

Step 5 — Visualise the Decision Tree

python
```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
plt.figure(figsize=(16, 6))
plot_tree(model, feature_names=['pclass', 'sex', 'age', 'fare'],
        class_names=['Not survived', 'Survived'],
        filled=True, rounded=True, fontsize=10)
plt.title("Decision Tree — Titanic Survival")
plt.tight_layout()
plt.savefig("decision_tree.png", dpi=150)
plt.show()
```

This plot shows the full tree — every question, every branch, and every prediction. Looking at the root node, the first split is on `sex`. This makes sense because historically, women and children were given priority during the evacuation. The tree discovered this pattern on its own just by looking at the data, which I found really interesting.

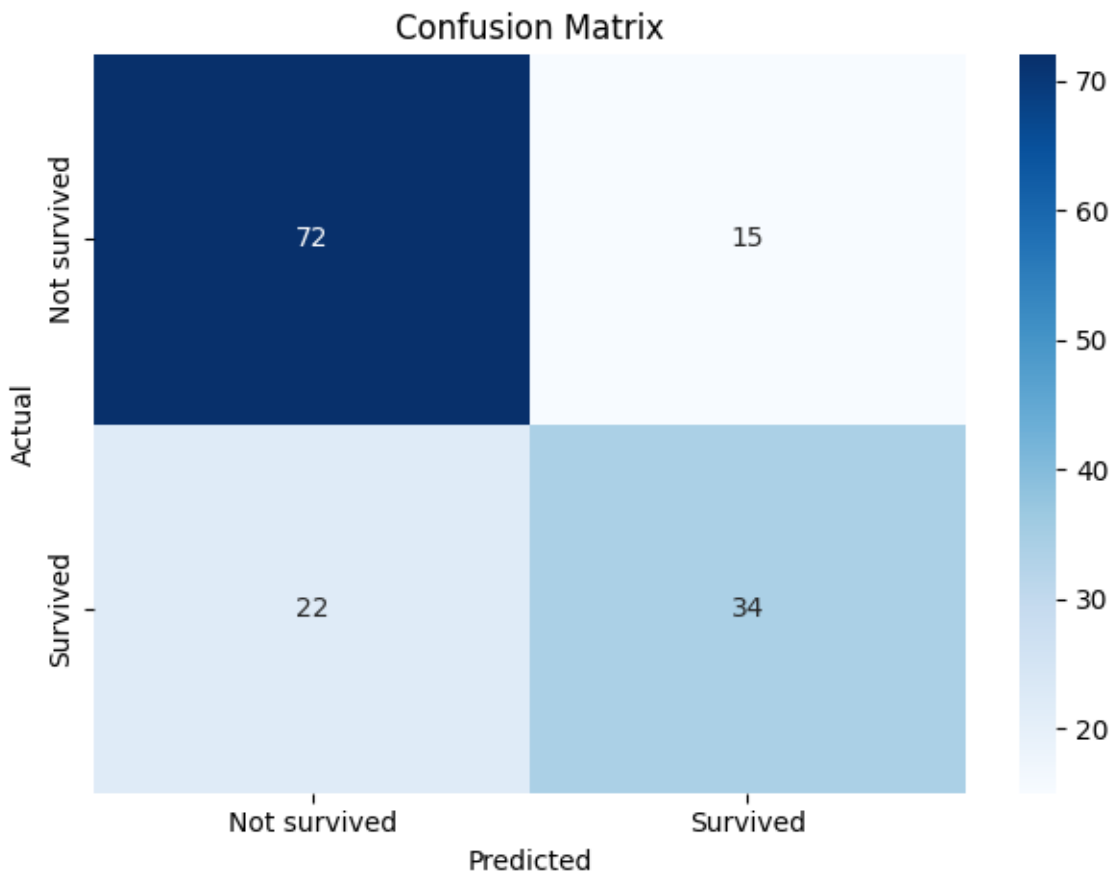


Step 6 — Confusion Matrix

python
```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
        xticklabels=['Not survived', 'Survived'],
        yticklabels=['Not survived', 'Survived'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=150)
plt.show()
```

The confusion matrix gives a clearer picture of where the model was right and where it was wrong. From my results:

- 72 passengers were correctly predicted as not survived
- 34 passengers were correctly predicted as survived
- 15 were predicted as survived but actually did not
- 22 were predicted as not survived but actually did



No model is 100% perfect, but these results are reasonable for a tree with only 3 levels of depth.

Conclusion

In this tutorial, I built a Decision Tree classifier to predict Titanic survival using Python and scikit-learn. I covered how to load and clean data, split it into training and test sets, train the model, evaluate it using accuracy and a confusion matrix, and visualise the tree.

What surprised me most was how the tree automatically identified gender as the most important feature —something that matches the real historical context. This shows that even a simple algorithm can pick up meaningful patterns from data.

References

- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Waskom, M. (2021). Seaborn: Statistical Data Visualization. *Journal of Open Source Software*, 6(60), 3021.
- Quinlan, J.R. (1986). Induction of Decision Trees. *Machine Learning*, 1(1), 81–106.
- Titanic dataset via seaborn: https://github.com/mwaskom/seaborn-data

Github Repository

https://github.com/boomiga18/decision-tree-titanic-classification